



**Mecanismo de detección automática de DERIVA ARQUITECTÓNICA
mediante el uso de Fitness Functions y análisis estático de grafos de
dependencia: Un enfoque de gobernanza continua para PyMES de
software.**

Juan José Medicis Zambrano

Anteproyecto presentado como requisito parcial para optar al título de:
Magister en Ingeniería de Software

Director(a):
Título (Ph.D., MSc) y nombre del director(a)

Pontificia Universidad Javeriana Cali
Facultad de Ingeniería y Ciencias
Departamento de Electrónica y Ciencias de la Computación
Cali, Colombia
28 de abril de 2026

Abstract

Architectural drift is one of the main causes of structural software deterioration in SMEs, since design decisions progressively diverge from the actual implementation without continuous verification mechanisms. This proposal aims to develop an automated mechanism for detecting architectural drift based on static code analysis, dependency graph construction, and the execution of *Fitness Functions*, in order to support technical governance without relying on external architectural documentation. The work will be limited to a single technological ecosystem and will evaluate three specific conformance dimensions: layer integrity, absence of dependency cycles, and compliance with architectural naming conventions. From these rules, the mechanism will generate a structural representation of the system and calculate a Conformance Rate (CR) to identify relevant deviations at an early stage. The expected outcomes include a functional prototype, a set of preconfigured rules, and evidence of the usefulness of the approach to support maintenance and refactoring decisions in agile development environments. In this way, the project seeks to contribute to a more continuous, objective, and feasible architectural governance practice for organizations with limited resources.

Resumen

La deriva arquitectónica representa una de las principales causas del deterioro estructural del software en PyMES, debido a que las decisiones de diseño se desalinean progresivamente de la implementación real sin que existan mecanismos continuos de verificación. Este anteproyecto propone desarrollar un mecanismo automatizado de detección de deriva arquitectónica basado en análisis estático de código, construcción de grafos de dependencia y ejecución de *Fitness Functions*, con el fin de apoyar la gobernanza técnica sin depender de documentación arquitectónica externa. El trabajo se enfocará en un único ecosistema tecnológico y evaluará tres dimensiones concretas de conformidad: integridad de capas, ausencia de ciclos de dependencia y cumplimiento de convenciones de nomenclatura arquitectónica. A partir de estas reglas, el mecanismo generará una representación estructural del sistema y calculará una Tasa de Conformidad (CR) que permita identificar desviaciones relevantes de manera temprana. Se espera como resultado un prototipo funcional, un conjunto de reglas preconfiguradas y evidencia sobre la utilidad del enfoque para apoyar decisiones de mantenimiento y refactorización en contextos de desarrollo ágil. Con ello, el proyecto busca contribuir a una práctica de gobernanza arquitectónica más continua, objetiva y viable para organizaciones con recursos limitados.

Índice

1. Planteamiento y Justificación del Problema	5
1.1. Planteamiento del Problema:	5
1.2. Justificación del Proyecto:	6
2. Objetivos del proyecto	6
2.1. Objetivo General	6
2.2. Objetivos específicos	6
3. Alcance del Proyecto	7
4. Marco teórico y antecedentes	8
4.1. Bases Teóricas	8
4.2. Antecedentes	10
5. Gestión y Mitigación de Riesgos Éticos	12
6. Metodología del proyecto	14
6.1. Diseño metodológico	14
6.2. Fases de ejecución	15
6.3. Técnicas e instrumentos de recolección y análisis	16
6.4. Criterios de cierre	16
7. Cronograma	17

Índice de figuras

Índice de tablas

1. Cronograma mensual propuesto para el anteproyecto	18
--	----

1. Planteamiento y Justificación del Problema

1.1. Planteamiento del Problema:

En el ecosistema actual de desarrollo de software, la arquitectura ha dejado de ser un plano estático para convertirse en una entidad dinámica que evoluciona en cada iteración. Este dinamismo da origen a la deriva arquitectónica (architecture drift), definida como la divergencia progresiva entre la arquitectura planeada (prescrita) y la arquitectura realmente implementada en el código fuente (actual) (Martín y col., s.f.).

La investigación indica que aproximadamente el 65 % del presupuesto de TI de una organización se consume exclusivamente en el mantenimiento de software, a menudo debido a problemas inducidos por la erosión arquitectónica. Esta erosión puede incrementar los costos de desarrollo entre un 20 % y 40 % en comparación con sistemas bien mantenidos. Sin embargo, el 80 % de las organizaciones ágiles y PyMES carecen de documentación arquitectónica actualizada, lo que invalida los métodos tradicionales de gobernanza basados en revisiones manuales o comités (Manglaras y col., 2026).

Sin mecanismos automatizados, la deriva introduce "smells" estructurales como dependencias cíclicas, componentes "Dios" (God Components) y violaciones de capas, que elevan el Índice de Deuda Arquitectónica (ADI) («Transforming enterprise architecture into a decision infrastructure – Djimit van data naar doen.» s.f.). Esto lleva al "Legacy Trap", donde la complejidad acumulada impide la innovación estratégica («Transforming enterprise architecture into a decision infrastructure – Djimit van data naar doen.» s.f.). Por tanto, existe una necesidad crítica de mecanismos que realicen una gobernanza continua basándose exclusivamente en el código fuente, transformando las reglas de diseño en Fitness Functions ejecutables («Make your architecture agile with fitness functions | Okoone», s.f.).

1.2. Justificación del Proyecto:

En ausencia de un mecanismo automatizado de detección de deriva arquitectónica, las PyMES de software continúan operando bajo un modelo reactivo de gobernanza técnica, donde la arquitectura solo se revisa cuando ya existen fallos visibles en el sistema. Esta falta de visibilidad estructural genera una acumulación progresiva de desviaciones entre la arquitectura diseñada y la implementada, fenómeno que no se manifiesta inicialmente como errores funcionales, sino como deterioro silencioso de la calidad interna del sistema (Jolak y col., 2025).

El proyecto propuesto no busca prevenir fallos visibles, sino evitar que el software se convierta en un sistema rígido incapaz de adaptarse al cambio — una condición que representa uno de los principales riesgos estratégicos para las PyMES en entornos digitales altamente dinámicos.

2. Objetivos del proyecto

2.1. Objetivo General

Desarrollar un mecanismo automatizado para la detección de derivas arquitectónicas, basado en grafos de dependencia y la ejecución de funciones de fitness, eliminando la dependencia de documentación externa para garantizar la integridad estructural en proyectos de software de PyMES.

2.2. Objetivos específicos

1. Definir un conjunto de Fitness Functions basadas en el análisis de smells arquitectónicos más comunes (acoplamiento eferente, ciclos y violación de capas) adaptadas a entornos de desarrollo ágil.
2. Implementar un componente de análisis estático que genere automáticamente grafos de dependencia y topologías de componentes a partir del código fuente y de archivos de configuración, usando una herramienta propietaria.

3. Construir un motor de validación que compare el grafo extraído con las reglas definidas, calculando una **Tasa de Conformidad (CR)** automatizada en ciertos aspectos definidos.
4. Evaluar el mecanismo en un proyecto en condiciones controladas, analizando su capacidad para identificar los smells arquitectónicos comunes y determinar su efectividad en proyectos de software de PyMES.

3. Alcance del Proyecto

- **Dominio Tecnológico:** El prototipo se limitará a un único ecosistema (Java o .NET) para garantizar la profundidad técnica dentro del tiempo establecido..
- **Reglas de Detección:** El análisis se enfocará en tres dimensiones: integridad de capas (ej. la capa de persistencia no debe llamar a la de presentación), ausencia de ciclos de dependencia y cumplimiento de convenciones de nomenclatura arquitectónica (Niessen, s.f.).
- **Entregables:** Un repositorio con el prototipo funcional, un catálogo de reglas pre-configuradas y el informe final de investigación.

4. Marco teórico y antecedentes

4.1. Bases Teóricas

Esta sección fundamenta los conceptos pilares sobre los cuales se construye el mecanismo de detección automatizada.

- **Deriva y Erosión Arquitectónica:** Se define como la divergencia progresiva entre la arquitectura prescrita (planeada) y la arquitectura realmente implementada en el código fuente. Este fenómeno implica una pérdida de integridad estructural debido a la violación de principios de diseño originales (Pigazzini, 2021). La investigación reciente destaca que esta divergencia es un problema persistente cuando no existen acciones de contención, impactando negativamente en la mantenibilidad y la capacidad de evolución del sistema (Anthony y col., 2024). Se estima que el costo del mantenimiento de software, a menudo inflado por la erosión, consume aproximadamente el 65 % del presupuesto de TI de una organización (Pigazzini, 2021).
- **Deuda Técnica Arquitectónica (ATD):** Es una metáfora que describe las decisiones de diseño subóptimas que ofrecen beneficios a corto plazo (como entregas rápidas) pero deterioran la calidad a largo plazo, generando un interés.^{en} forma de mayor esfuerzo de mantenimiento (Hamed y col., 2023). La ATD se diferencia de la deuda a nivel de código porque involucra decisiones estructurales globales como capas, subsistemas e interfaces (Pigazzini, 2021). Un síntoma clave de su acumulación es la presencia de Smells Arquitectónicos (AS) (Sas & Avgeriou, 2022).
- **Smells Arquitectónicos (AS):** Son decisiones de diseño que impactan negativamente en la calidad interna del sistema (Pigazzini, 2021). A diferencia de los code smells, que ocurren a nivel de métodos o clases, los AS involucran múltiples componentes, paquetes o capas (Sas & Avgeriou, 2022). Ejemplos críticos incluyen la **Dependencia Cíclica (CD)**, donde los componentes no pueden ser liberados o probados de forma aislada, y el **Componente Dios (GC)**,

que ocurre cuando una entidad es excesivamente grande y difícil de entender (Pigazzini, 2021; Sas & Avgeriou, 2022).

- **Funciones de Aptitud (Fitness Functions):** Una función de aptitud es un mecanismo diseñado para resumir qué tan cerca está una solución de diseño específica de alcanzar sus objetivos y preservar sus características clave. Se definen formalmente como "cualquier mecanismo que proporciona una evaluación de integridad objetiva de alguna característica arquitectónica". Su propósito fundamental es actuar como barandillas (guardrails) automatizadas que comunican, validan y preservan atributos de calidad (requisitos no funcionales) de manera continua, asegurando que la evolución del sistema se mantenga dentro de un rango y una dirección deseados por los arquitectos. Esta disciplina permite a los equipos ser reactivos ante la degradación de características críticas en tiempo real, evitando que la realidad del sistema en producción se desvíe de las intenciones de diseño originales («Fitness Functions – Safeguard Architecture with Automated Checks», s.f.).
- **Análisis Estático y Grafos de Dependencia:** Es la técnica de extraer hechos y abstracciones del código fuente sin ejecutarlo (Pigazzini, 2021). Los grafos de dependencia representan los componentes del sistema como nodos y sus relaciones como aristas, permitiendo aplicar algoritmos de búsqueda (como *Depth First Search*) para detectar estructuras problemáticas como ciclos (Klein y col., 2022; Pigazzini, 2021).
- **Algoritmos de Recorrido sobre el Grafo de Dependencias:** La detección de smells arquitectónicos sobre el grafo $G = (V, E)$ requiere la aplicación de algoritmos de grafos dirigidos. La **Dependencia Cíclica (CD)** se detecta mediante una búsqueda en profundidad (*DFS*) que identifica Componentes Fuertemente Conexos (SCC), clasificando un subconjunto de nodos como cíclico cuando $|SCC| > 1$. La **Dependencia Inestable (UD)** se calcula midiendo el acoplamiento aferente (C_a = aristas entrantes) y eferente (C_e = aristas salientes) de cada nodo, derivando la métrica $I = C_e / (C_a + C_e)$; una relación

$A \rightarrow B$ constituye un UD cuando el componente estable A depende del inestable B ($I_A < I_B$). El **Componente Dios (GC)** se detecta cuando el número de dependencias de salida de un paquete supera un umbral configurable. Sas y col. (2019) validaron empíricamente estas métricas analizando la evolución de smells de inestabilidad en proyectos Java de código abierto mediante el recorrido sistemático del grafo de dependencias a nivel de paquete (Sas y col., 2019).

4.2. Antecedentes

La investigación actual presenta diversas estrategias para gestionar la integridad técnica del software, evolucionando desde la medición manual hacia enfoques predictivos y automatizados.

- **ArchUnit:** Es una biblioteca gratuita y extensible diseñada para verificar la arquitectura de código Java mediante cualquier entorno de pruebas unitarias estándar. Funciona analizando el bytecode e importando las clases en una estructura de código Java, lo que permite comprobar dependencias entre paquetes, capas, ciclos y más. Es la herramienta que permite materializar el concepto de "arquitectura auto-testeable" al permitir escribir reglas arquitectónicas como si fueran pruebas unitarias comunes («Unit test your Java architecture - ArchUnit», s.f.).
- **ArchSync:** Este es una herramienta de asistencia que busca mantener en sincronía el código con los escenarios arquitectónicos (Use Case Maps). ¿Qué hace?, Utiliza heurísticas para procesar trazas de ejecución y correlacionarlas con el comportamiento arquitectónico esperado, sugiriendo "scripts de actualización" para corregir inconsistencias. Aunque ArchSync se apoya más en el comportamiento dinámico (trazas), coincide en reducir el esfuerzo de sincronización manual entre arquitectos y desarrolladores. (Diaz-Pace y col., 2011)
- **Gestión en PyMES:** (Hamed y col., 2023) proponen un enfoque de gestión de deuda técnica diseñado específicamente para pequeñas y medianas empresas

(SMEs). Destacan que las limitaciones de recursos, presupuestos ajustados y la falta de documentación actualizada en estos entornos hacen vital una gobernanza basada en herramientas de análisis estático (como SonarQube) que operen sobre el artefacto vivo del software (Hamed y col., 2023).

- **Índices de Deuda Basados en Machine Learning:** (Sas & Avgeriou, 2022) desarrollaron el **ATDI (Architectural Technical Debt Index)**, el cual utiliza modelos de aprendizaje automático (gradient boosting) para clasificar la severidad de los smells arquitectónicos. Este enfoque combina el análisis estático de las líneas de código afectadas con la severidad estimada, logrando un 71 % de acuerdo con la percepción de esfuerzo de refactorización de desarrolladores expertos (Sas & Avgeriou, 2022).
- **Conformidad Automatizada en el Flujo de CI/CD:** (Klein y col., 2022) subrayan que el uso de verificadores de conformidad automatizados integrados en los flujos de Integración Continua (CI) permite exponer las desviaciones en el momento del commit. Esto facilita la remediación inmediata antes de que las violaciones se fijen en la implementación, evitando que la deriva se convierta en un problema de mantenimiento costoso meses o años después (Klein y col., 2022).
- **Modelo de Reflexión (Software Reflexion Models):** Murphy y col. (2001) introdujeron el mecanismo formal para cuantificar la divergencia entre el grafo de dependencias del código fuente (*source model*) y la arquitectura prescrita (*high-level model*), clasificando cada relación en convergencias (permitidas), divergencias (violaciones = deriva) o ausencias (esperadas pero faltantes). Esta tripartición fundamenta directamente el motor de validación y la Tasa de Conformidad (CR) de este trabajo (Murphy y col., 2001).

5. Gestión y Mitigación de Riesgos Éticos

El valor del proyecto no depende únicamente de identificar riesgos éticos, sino de establecer desde el diseño del prototipo un conjunto de controles que reduzcan la probabilidad de daño y limiten el uso indebido de sus resultados. En ese sentido, la estrategia ética del anteproyecto se fundamenta en cuatro principios: privacidad por diseño, transparencia del diagnóstico, supervisión humana y uso proporcional de las métricas.

- **Mitigación del riesgo sobre información sensible:** El análisis estático requiere acceso al código fuente, por lo que la primera medida de control consiste en operar el prototipo en entornos restringidos y bajo acuerdos explícitos de confidencialidad. Los artefactos generados deberán limitarse a relaciones estructurales y métricas agregadas, evitando exponer fragmentos de código, credenciales o detalles de lógica de negocio. Asimismo, se priorizará el procesamiento local de los repositorios y la minimización de datos almacenados, de manera que el mecanismo observe la arquitectura sin convertir el código fuente en un insumo reutilizable por terceros (Hamed y col., 2023; Modawski & Wolniak, 2025).
- **Mitigación del sesgo algorítmico y de la opacidad del diagnóstico:** Para evitar decisiones de refactorización basadas en umbrales arbitrarios o interpretaciones erróneas, las reglas de evaluación deberán ser explícitas, versionadas y trazables a criterios arquitectónicos previamente definidos. En lugar de presentar la herramienta como una caja negra, cada resultado deberá explicar qué regla se activó, cuál fue la evidencia estructural encontrada y qué severidad se asignó. Además, la validación del prototipo se realizará con revisión humana sobre casos de prueba controlados, de forma que la salida automatizada se utilice como apoyo a la decisión y no como veredicto incuestionable (Klein y col., 2022; Sas & Avgeriou, 2022).
- **Mitigación del impacto sociotécnico en los equipos:** Para reducir la posibilidad de que la gobernanza automatizada sea percibida como vigilancia

punitiva, el uso de los reportes deberá orientarse al nivel de componente o sistema y no a la evaluación individual de desarrolladores. El mecanismo se incorporará como soporte para conversaciones técnicas, retrospectivas arquitectónicas y priorización de refactorización, no como instrumento disciplinario. Esta delimitación de uso debe quedar explícita en la metodología del proyecto y en la interpretación de resultados, protegiendo la moral del equipo y favoreciendo la adopción de la herramienta como ayuda de mejora continua (Hamed y col., 2023; Paul & Wang, 2019).

- **Mitigación del formalismo vacío:** Existe el riesgo de que las *Fitness Functions* se conviertan en una rutina burocrática sin reflexión arquitectónica real. Para evitarlo, los hallazgos del prototipo deberán complementarse con revisión contextual por parte del equipo, registro de excepciones justificadas y ajustes periódicos de las reglas conforme evolucione el sistema. La métrica de conformidad no debe reemplazar el juicio técnico, sino activar discusiones de diseño informadas y verificables (Modawski & Wolniak, 2025).
- **Mitigación del riesgo de sobreinterpretación de la Tasa de Conformidad (CR):** La CR producida por el mecanismo solo representa el cumplimiento de las reglas configuradas dentro del alcance del prototipo: integridad de capas, ausencia de ciclos de dependencia y convenciones de nomenclatura. Para prevenir una falsa sensación de control, cada reporte deberá declarar de manera visible qué reglas fueron evaluadas, qué módulos fueron analizados, qué umbrales se aplicaron y qué aspectos quedaron fuera del diagnóstico. En consecuencia, el prototipo se presentará como una herramienta de detección temprana y no como un árbitro definitivo de la calidad arquitectónica global (Klein y col., 2022; Modawski & Wolniak, 2025).

En conjunto, estas medidas buscan que la automatización propuesta mantenga valor técnico sin desbordar su propósito investigativo ni generar incentivos de uso contrarios a la ética profesional. La mitigación, por tanto, no se plantea como un anexo del proyecto, sino como una condición de validez para su aplicación en contextos reales de PyMES de software.

6. Metodología del proyecto

Este proyecto se desarrollará como una investigación aplicada, con enfoque predominantemente cuantitativo y con estrategia de **Design Science Research (DSR)**, dado que el resultado principal es la construcción y evaluación de un artefacto software orientado a resolver un problema práctico de gobernanza arquitectónica (Hamed y col., 2023; Klein y col., 2022). Para que el trabajo sea alcanzable dentro de los cinco meses y las 192 horas definidas para el anteproyecto, la estrategia DSR se ejecutará mediante ciclos iterativos cortos, cada uno con un entregable verificable y un cierre de revisión con el director.

6.1. Diseño metodológico

El diseño metodológico se apoyará en las siguientes decisiones operativas:

- **Unidad de trabajo:** un único sistema de software con código fuente disponible, arquitectura en capas y tamaño acotado, implementado en un solo ecosistema tecnológico seleccionado al inicio del proyecto (Java o .NET).
- **Artefacto a construir:** un prototipo que extraiga dependencias desde el código fuente, construya un grafo estructural, ejecute reglas de validación arquitectónica y produzca un reporte con hallazgos y una **Tasa de Conformidad (CR)**.
- **Alcance técnico del prototipo:** el mecanismo se limitará a tres dimensiones de conformidad: integridad de capas, ausencia de ciclos de dependencia y cumplimiento de convenciones de nomenclatura arquitectónica. Esta delimitación mantiene el esfuerzo dentro del tiempo disponible y es coherente con los objetivos específicos del anteproyecto (Anthony y col., 2024; Niessen, s.f.).
- **Estrategia de validación:** el artefacto se evaluará mediante un estudio de caso controlado, ejecutándolo primero sobre una línea base y después sobre versiones con derivas introducidas deliberadamente para verificar su capacidad de detección (Pigazzini, 2021; Sas & Avgeriou, 2022).

6.2. Fases de ejecución

1. **Delimitación del caso de estudio y catálogo de reglas:** se seleccionará el ecosistema tecnológico a trabajar, se escogerá el sistema base y se definirá un catálogo acotado de reglas ejecutables. Como resultado se construirá una matriz de validación con la descripción de cada regla, su propósito, la evidencia esperada y el criterio de incumplimiento. En esta fase se tomarán como referencia las *Fitness Functions* como mecanismo para traducir decisiones arquitectónicas en verificaciones automáticas («Fitness Functions – Safeguard Architecture with Automated Checks», s.f.; Paul & Wang, 2019).
2. **Diseño técnico del prototipo:** se definirá la arquitectura interna del mecanismo, las entradas y salidas del proceso, la estructura del grafo de dependencia y el formato del reporte de resultados. El objetivo de esta fase es reducir riesgo de implementación antes de programar, dejando explícitas las clases de relaciones que se van a analizar y las restricciones que no formarán parte del alcance.
3. **Construcción del componente de extracción:** se implementará el módulo encargado de inspeccionar el código fuente y los archivos de configuración necesarios para inferir dependencias estructurales. El producto de esta fase será una representación navegable del sistema en forma de grafo, suficiente para consultar paquetes, componentes y relaciones relevantes para el análisis arquitectónico (Klein y col., 2022; Murphy y col., 2001).
4. **Construcción del motor de validación:** sobre el grafo obtenido se implementarán las reglas definidas en la primera fase. Este motor calculará la **CR** como proporción de reglas cumplidas frente al total de reglas evaluadas y reportará las violaciones detectadas por tipo, ubicación y severidad. Con ello se dará cumplimiento al objetivo de automatizar la verificación de conformidad arquitectónica (Paul & Wang, 2019; «Unit test your Java architecture - ArchUnit», s.f.).
5. **Evaluación y análisis de resultados:** el prototipo se ejecutará sobre el caso de estudio seleccionado y sobre versiones con derivas controladas, tales como

dependencias entre capas no permitidas, ciclos entre módulos y nomenclatura inconsistente. Los resultados se analizarán comparando las derivas esperadas con las efectivamente detectadas, la variación de la **CR** y la utilidad del reporte para apoyar decisiones de mantenimiento y refactorización (Anthony y col., 2024; Sas y col., 2019).

6.3. Técnicas e instrumentos de recolección y análisis

Para desarrollar y evaluar el prototipo se utilizarán las siguientes técnicas e instrumentos:

- **Revisión bibliográfica focalizada**, para consolidar criterios de deriva arquitectónica y seleccionar reglas viables para un prototipo de alcance acotado.
- **Matriz de reglas arquitectónicas**, en la que se registrará cada regla, su racionalidad, patrón esperado y evidencia de incumplimiento.
- **Bitácora de desarrollo y ejecución**, donde se documentarán decisiones, ajustes del prototipo, resultados de las corridas y observaciones del director.
- **Tablas comparativas de resultados**, orientadas a registrar número de violaciones detectadas, porcentaje de conformidad, tipo de deriva identificada y diferencias entre la línea base y las versiones alteradas.

6.4. Criterios de cierre

Se considerará que la metodología ha producido resultados suficientes para el anteproyecto cuando se cumplan los siguientes criterios: a) existencia de un catálogo acotado de reglas ejecutables, b) disponibilidad de un prototipo funcional sobre un único ecosistema, c) generación de reportes de conformidad a partir del código fuente y d) evidencia de que el mecanismo identifica derivas introducidas en un entorno controlado. Estos criterios permiten mantener una meta realista y medible, adecuada al tiempo de dedicación previsto para el trabajo.

7. Cronograma

El proyecto se plantea para ejecutarse en **cinco meses**, con una dedicación promedio de **9 a 10 horas por semana**, para un total de **192 horas**. El cronograma adopta una lógica iterativa: cada mes cierra con una revisión de avance, ajuste del backlog técnico y validación de entregables con el director.

Mes	Periodo	Actividades principales	Horas	Entregable
Mes 1	Mayo 2026	Revisión bibliográfica focalizada, selección del ecosistema y del caso de estudio, definición del catálogo inicial de reglas, construcción de la matriz de validación y primera revisión con el director.	34	Protocolo de trabajo y catálogo inicial de reglas
Mes 2	Junio 2026	Diseño técnico del prototipo, definición de entradas y salidas, modelado del grafo de dependencia, preparación del entorno de desarrollo y construcción del extractor base.	38	Diseño del prototipo y extracción inicial de dependencias
Mes 3	Julio 2026	Implementación del componente de análisis estático, consolidación del grafo estructural, codificación de reglas de capas y nomenclatura, pruebas iniciales sobre el caso base.	42	Prototipo funcional parcial con primeras reglas ejecutables
Mes 4	Agosto 2026	Implementación de detección de ciclos, cálculo de la Tasa de Conformidad (CR), generación de reportes, inyección controlada de derivas y ajustes del prototipo a partir de los resultados.	40	Prototipo funcional completo y reporte preliminar de conformidad
Mes 5	Septiembre 2026	Ejecución final del estudio de caso, análisis comparativo entre línea base y versiones alteradas, consolidación de resultados, redacción del informe de avances y preparación de la socialización.	38	Resultados analizados, conclusiones preliminares y documento consolidado
Total			192	

Tabla 1: Cronograma mensual propuesto para el anteproyecto

Para mantener el cronograma bajo control, al cierre de cada mes se realizará una

revisión de entre 1 y 2 horas con el director. Esa revisión permitirá contrastar horas planeadas frente a horas reales, depurar el alcance si aparece sobrecarga y priorizar el siguiente ciclo de trabajo sin comprometer los objetivos centrales del proyecto.

La distribución de horas por tipo de trabajo quedará así: **32 h** para revisión y definición de reglas, **80 h** para diseño e implementación del prototipo, **42 h** para evaluación controlada y ajustes, y **38 h** para documentación, reuniones y cierre. Esta distribución concentra el mayor esfuerzo en la construcción del artefacto, pero conserva tiempo suficiente para validar resultados y documentar el proceso de manera académicamente defendible.

Referencias

- Anthony, E., Berntsson, A., Santilli, T. & Wohlrab, R. (2024). We're Drifting Apart: Architectural Drift from the Developers' Perspective. *Proceedings of the IEEE 21st International Conference on Software Architecture (ICSA 2024)*, 101-111. <https://doi.org/10.1109/ICSA59870.2024.00018>
- Diaz-Pace, J. A., Soria, A., Rodriguez, G. & Campo, M. R. (2011). Taking ArchSync to the Real World: An Analysis of Three Case-Studies.
- Fitness Functions – Safeguard Architecture with Automated Checks. (s.f.). <https://continuous-architecture.org/practices/fitness-functions/>
- Hamed, A. S. E., Elbakry, H. M., Riad, A. E. & Moawad, R. (2023). A Proposed Technical Debt Management Approach Applied on Software Projects in Egypt. *Journal of Internet Services and Information Security (JISIS)*, 13(3), 156-177. <https://doi.org/10.58346/JISIS.2023.I3.010>
- Jolak, R., Karlsson, S. & Dobslaw, F. (2025). An empirical investigation of the impact of architectural smells on software maintainability. *Journal of Systems and Software*, 225, 112382. <https://doi.org/10.1016/j.jss.2025.112382>
- Klein, J., Nord, R. & Shull, F. (2022). *Why Architecture Conformance Matters for Software Systems* (inf. téc. N.º DM22-0924). Software Engineering Institute, Carnegie Mellon University.
- Make your architecture agile with fitness functions | Okoone. (s.f.). <https://www.okoone.com/spark/technology-innovation/make-your-architecture-agile-with-fitness-functions/>
- Manglaras, O., Farkas, A., Woolford, T., Treude, C. & Wagner, M. (2026). ModARO: A Modular Approach to Architecture Reconstruction of Distributed Microservice Codebases. <https://arxiv.org/pdf/2602.08181v1>
- Martín, D. S., Camargo, V. V. D. & Angulo, G. (s.f.). Architectural Conformance Checking for MAPE-K-based Self-Adaptive Systems.
- Modawski, A. & Wolniak, A. (2025). Lightweight AI Governance (LAIG) Framework for SMEs. *Proceedings of the European Workshop on Trustworthy AI*

- (*TRUST-AI 2025*), organized as part of the *European Conference of Artificial Intelligence (ECAI 2025)*. <https://ceur-ws.org/Vol-4132/short57.pdf>
- Murphy, G. C., Notkin, D. & Sullivan, K. J. (2001). Software Reflexion Models: Bridging the Gap between Source and High-Level Models. *IEEE Transactions on Software Engineering*, 27(4), 364-380. <https://doi.org/10.1109/32.908994>
- Niessen, L. (s.f.). Fitness Functions: Automating Your Architecture Decisions | by Lukas Niessen | Feb, 2026 | Medium. <https://lukasniessen.medium.com/fitness-functions-automating-your-architecture-decisions-08b2fe4e5f34>
- Paul, P. & Wang, R. (2019). Fitness function-driven development. *Thoughtworks*. <https://www.thoughtworks.com/insights/articles/fitness-function-driven-development>
- Pigazzini, I. (2021). *Evaluating and Detecting Architecture Erosion* (PhD Thesis). Università degli Studi di Milano-Bicocca.
- Sas, D. & Avgeriou, P. (2022). An architectural technical debt index based on machine learning and architectural smells [Bernoulli Institute, University of Groningen].
- Sas, D., Avgeriou, P. & Fontana, F. A. (2019). Investigating Instability Architectural Smells Evolution: An Exploratory Case Study. *2019 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 557-567. <https://doi.org/10.1109/ICSME.2019.00090>
- Transforming enterprise architecture into a decision infrastructure – Djimit van data naar doen. (s.f.). <https://djimit.nl/transforming-enterprise-architecture-into-a-decision-infrastructure/>
- Unit test your Java architecture - ArchUnit. (s.f.). <https://www.archunit.org/>